Лабораторная работа 7. Наблюдаемость и регрессия: трейсинг и регрессионный гейт в CI

Главный вопрос. Как заметить, что изменение сломало задачи, которые система раньше решала, и найти место поломки?
Что получится. Трейсинг агента в MLflow, регрессионный набор и правило принятия изменений (гейт), устойчивое к случайному разбросу; конвейер CI с тестами без модели и проверкой гейта; опыт с реалистичным изменением и проверка ложных тревог; отчёт-дашборд.
Среда. Агент и оценочный контур ЛР4–ЛР5, AgentDojo (`workspace`), MLflow 3, GitHub Actions или GitLab CI, сервер модели из ЛР1 или внешний API.

1. Зачем эта работа

Агентную систему постоянно меняют: правят системный промпт, переходят на новую модель, квантуют её ради скорости, обрезают длинные результаты инструментов, чтобы сэкономить токены. Каждое изменение может поднять среднюю успешность и при этом сломать конкретные задачи, которые раньше решались. В обычной разработке такие поломки ловят регрессионные тесты; для ML-систем к ним добавляют проверки поведения на зафиксированных сценариях и наблюдение за работой [1, 2].

У агентов две особенности. Первая — шум: из ЛР5 вы знаете, что задача может провалиться случайно, и тест, который иногда падает без причины, быстро перестают замечать. Вторая — поиск причины: если задача сломалась, нужно увидеть, на каком вызове модели или инструмента поведение разошлось с прежним. Для этого нужен **трейсинг** (tracing) — подробная запись каждого эпизода по шагам.

В этой работе вы добавите в агента трейсинг, определите регрессионный набор и правило, по которому изменение принимается или отклоняется, запустите проверку в CI и испытаете всё на реалистичном изменении. Отдельно вы проверите, не поднимает ли правило ложную тревогу, когда система вообще не менялась.

2. Как устроено

2.1. Версия системы

Поведение агента определяют код, системный промпт и промпты плана, описания инструментов, модель (имя, ревизия, квантование), параметры генерации и лимиты, а также версия стенда. Всё это вместе — **версия системы**. Её описывает манифест: коммит, SHA-256 файлов промптов и конфигурации, сведения о модели и сервере. Любое изменение любой части — новая версия, даже если код не менялся.

2.2. Трейсинг

Трейс (trace) — запись одного эпизода; он состоит из **спанов** (span). Спан — одна операция: весь эпизод, вызов модели, вызов инструмента. У спана есть начало и конец, родительский спан, статус и атрибуты: модель, число токенов, имя инструмента, аргументы. Такая модель данных принята в OpenTelemetry [3]; для вызовов языковых моделей там же описаны стандартные атрибуты, например `gen_ai.request.model` и `gen_ai.usage.input_tokens` [4].

MLflow Tracing [5] записывает трейсы в ту же базу `mlflow.db`, что и запуски:

- `mlflow.openai.autolog()` автоматически создаёт спан на каждый вызов клиента `openai`, в том числе к собственному серверу с OpenAI-совместимым API, и записывает в него токены;
- спаны эпизода и инструментов создаются вручную: `mlflow.start_span(name=..., span_type=...)` или декоратор `@mlflow.trace`;

- трейсы смотрят во вкладке Traces интерфейса MLflow или получают таблицей через `mlflow.search_traces()`.

Статус спана и успех задачи — разные вещи: вызов модели может завершиться без ошибок и вернуть неверный ответ. Поэтому итог проверки стенда записывается тегом трейса. И одна ловушка: MLflow отправляет трейсы в хранилище асинхронно, поэтому при чтении сразу после эпизода нужен параметр `flush=True`.

2.3. Регрессионный набор и гейт

Сравниваются две версии: **база** — принятая версия системы, и **кандидат** — версия с изменением. Обе прогоняются на 30 итоговых задачах по три раза при температуре 0.

- **Защищаемые задачи** — те, что база решила во всех трёх прогонах. Задачи, которые база решает через раз, для гейта бесполезны: их «поломка» чаще всего окажется шумом.
- Защищаемая задача считается **потерянной**, если кандидат решил её меньше чем в двух прогонах из трёх. Правило большинства делает гейт терпимым к одиночной случайной неудаче.

Решение гейта — один из четырёх статусов:

Статус	Когда
`FAIL`	Потеряна хотя бы одна защищаемая задача или не прошли тесты без модели
`REVIEW`	Потерь нет, но средняя успешность на остальных задачах упала на целую задачу и больше, медиана затрат или времени эпизода выросла больше чем на 25% или защищаемых задач меньше пяти, и гейт почти ничего не защищает
`PASS`	Ни одно условие выше не выполнено
`ERROR`	Сравнение невозможно: нет результатов, разные списки задач или версии стенда

Улучшения на других задачах потерю защищаемой задачи не компенсируют. Это сознательное правило: пользователь, у которого перестало работать то, что работало, не утешится тем, что заработало что-то другое. Пороги в таблице — учебные; в отчёте обоснуйте, подходят ли они вашей системе.

2.4. Где выполняется что

CI (continuous integration, непрерывная интеграция) — автоматический запуск проверок при каждом коммите [6]. Прогоны модели требуют GPU или API, а серверы CI обычно работают без GPU. Поэтому работа разделена:

- **оценка** выполняется там, где есть модель: скрипт `eval.py` прогоняет версию на задачах, пишет трейсы в MLflow и сохраняет компактную сводку `summary.json` — исход каждого прогона, токены, время, id трейса. Сводка небольшая и коммитится в репозиторий вместе с манифестом версии;
- **CI** при каждом коммите запускает тесты без модели и скрипт `gate.py`, который сравнивает сводки базы и кандидата и завершается ошибкой при `FAIL` или `ERROR`.

По желанию CI может ещё прогнать несколько отладочных задач через внешний API. Ключ тогда хранится в секретах CI [6, 7], а не в репозитории.

2.5. Отчёт-дашборд

`gate.py` строит отчёт в Markdown или HTML: версии и их манифесты; успешность, затраты и время обеих версий; списки потерянных и новых решённых задач со ссылками на трейсы; статус и его причина. Удобно сгруппировать показатели по «четырём золотым сигналам» наблюдения за сервисами — задержка, нагрузка, ошибки, насыщение [8]. Для агента это время эпизода, число вызовов модели и инструментов, доля ошибок инструментов и срабатываний лимитов, число токенов и заполнение контекста.

Рис. 1 показывает, как связаны части работы.

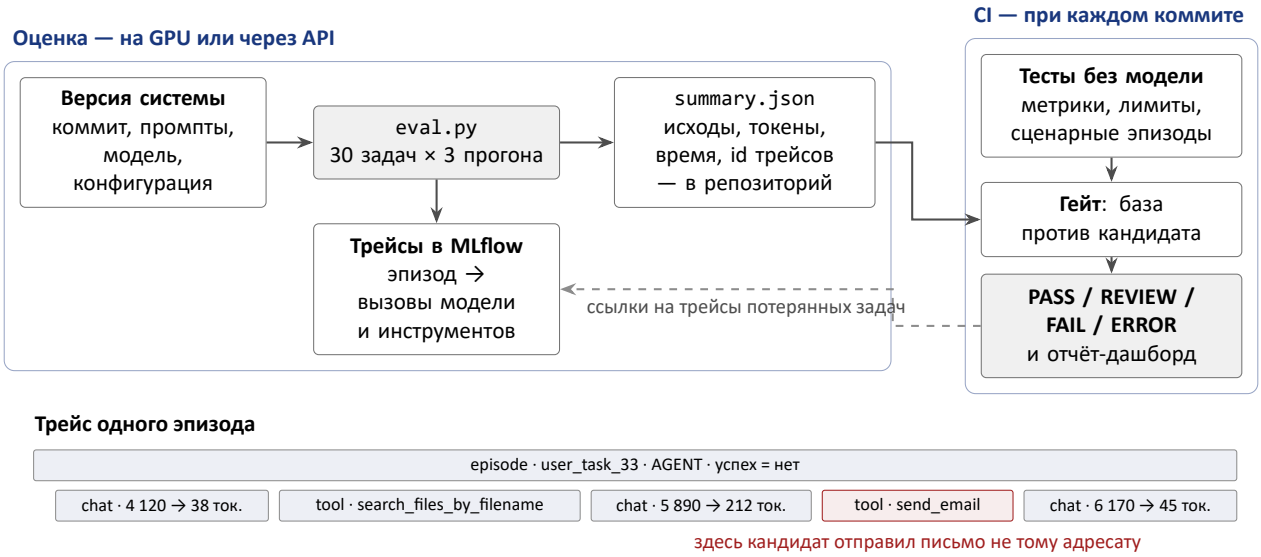


Рис. 1. Оценка версии, трейсы, гейт в CI и дерево спанов одного эпизода

3. Что сделать

3.1. Манифест версии и трейсинг

1. Манифест версии: коммит, SHA-256 промптов, описаний инструментов и конфигурации, модель и сервер, версия стенда. Сводка каждой оценки ссылается на манифест.
2. Трейсинг: спан эпизода типа `AGENT`, спаны вызовов модели (через `mlflow.openai.autolog()` или вручную), спаны инструментов типа `TOOL` с именем, аргументами, длиной результата и ошибкой. Теги трейса: задача, версия, номер прогона, итог проверки стенда.
3. В мультиагентном варианте из ЛР6 поручения исполнителям — отдельные спаны, внутри которых вложены спаны их вызовов.

3.2. Оценка, гейт и CI

1. `eval.py --version <имя> --runs 3` — прогоны версии на 30 итоговых задачах и `summary.json`.
2. `gate.py base.json candidate.json` — статус по п. 2.3, отчёт по п. 2.5 и код завершения.
3. Конфигурация CI (GitHub Actions или GitLab CI): установка зависимостей, тесты без модели, проверка гейта на сводках последней принятой версии и кандидата. Пример — в приложении.

3.3. Проверки без модели

- **Логика гейта на синтетических сводках:** одна потеря и одно улучшение — `FAIL`; нет потерь, но у базы три защищаемые задачи — `REVIEW`; у кандидата не хватает прогона одной задачи — `ERROR`.
- **Отрицательный контроль:** намеренно сломайте обвязку — например, исполнитель теряет результат последнего вызова инструмента — и прогоните сценарного агента из ЛР4 на задачах, которые он проходил. Сводка сломанной версии должна давать `FAIL`. Если гейт не замечает заведомую поломку, он бесполезен.
- Дерево спанов одного эпизода совпадает с последовательностью шагов в вашем трейсе ЛР4: тот же порядок вызовов, те же токены.

4. Эксперименты

До запусков выберите изменение для эксперимента 3 и запишите прогноз: какие задачи оно сломает и почему, как изменятся затраты, какой статус выдаст гейт.

4.1. Эксперимент 1. Цена трейсинга

На 10 отладочных задачах выполните по одному прогону с трейсингом и без него. Сравните медиану времени эпизода. Число токенов и исходы задач должны совпасть: трейсинг не должен менять поведение агента.

4.2. Эксперимент 2. База и проверка на ложную тревогу

1. Прогоните базовую версию — агента ЛР4 в режиме ReAct с конфигурацией ЛР5 — три раза на 30 итоговых задачах. Это сводка базы; определите защищаемые задачи.
2. **A/A-проверка**: прогоните ту же версию ещё три раза и сравните её с базой гейтом. Система не менялась, поэтому ожидается `PASS`. Если гейт выдал `FAIL` или `REVIEW`, разберите, какие задачи «сломались» и почему; это оценка того, как часто гейт будет тревожить зря.

4.3. Эксперимент 3. Реалистичное изменение

Выберите одно изменение:

- из системного промпта убрано одно правило, например требование краткого ответа или порядок действий при ошибке инструмента;
- модель заменена на меньшую или на квантованную версию (ЛР1);
- результаты инструментов обрезаются до 2000 символов;
- из описаний инструментов убраны описания параметров.

Прогоните кандидата три раза, получите статус гейта и отчёт. Для каждой потерянной задачи найдите в трейсах базы и кандидата **первое расхождение**: на каком вызове модели или инструмента поведение стало другим и почему. Если потерь нет, разберите задачу, у которой сильнее всего выросли затраты.

4.4. Эксперимент 4. Исправление

Попробуйте исправить регрессию, сохранив смысл изменения: например, оставить обрезку результатов, но добавить пометку «результат обрезан, уточните запрос». Прогоните исправленного кандидата и снова получите статус гейта. Если регрессий не было, выберите второе изменение из списка.

4.5. По желанию

- Экспорт трейсов по OTLP в Jaeger, Phoenix или Langfuse.
- Добавьте в гейт проверку безопасности: несколько атакованных пар из ЛР8 с требованием «ASR не вырос».
- Ночной запуск оценки по расписанию и сравнение с последней принятой версией.

5. Что сдать

- Код трейсинга, `eval.py`, `gate.py`, тесты, конфигурация CI; манифесты версий и сводки `summary.json`.
- Ссылка на успешный и неуспешный запуск конвейера CI или их журналы.
- Отчёты-дашборды для A/A-проверки, изменения и исправления.
- Отчёт на 3–4 страницы: прогноз; цена трейсинга; A/A-проверка; изменение, статус гейта и разбор первых расхождений; исправление; выводы о том, насколько гейт полезен вашей системе; кодогенерация.

Регрессий может не оказаться, и это допустимый результат. Ошибки постановки здесь чаще всего такие: база и кандидат прогнаны на разных задачах или разных версиях стенда; удачный прогон кандидата выбран из нескольких; пороги гейта подобраны после просмотра результатов; секреты в репозитории.

Вопросы к защите

1. Одна защищаемая задача потеряна, две новые решены. Почему гейт выдаёт `FAIL`, хотя средняя успешность выросла?
2. Зачем защищаются только задачи, которые база решила во всех трёх прогонах? Что было бы, если бы защищались все решённые хотя бы раз?
3. A/A-проверка выдала `REVIEW`. Что это говорит о системе и о гейте?
4. Покажите в трейсе первое расхождение базы и кандидата для потерянной задачи. Почему именно этот спан?
5. Чем статус спана отличается от успеха задачи? Приведите пример из своих трейсов.
6. Почему оценка с моделью не выполняется в CI при каждом коммите? Что тогда проверяет CI?
7. Уберите правило большинства и считайте задачу потерянной при любом неуспешном прогоне кандидата. Предскажите, что станет с A/A-проверкой, и пересчитайте по сохранённым сводкам.

Источники

1. Breck E. et al. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. IEEE Big Data, 2017. Статья — какие тесты и какой мониторинг нужны ML-системе (разд. 2–5).
2. Sculley D. et al. Hidden Technical Debt in Machine Learning Systems. NeurIPS, 2015. Статья — почему изменения в ML-системах ломают неочевидные вещи: переплетение компонентов, долг конфигурации (разд. 2–6).
3. OpenTelemetry. Traces — спаны, родительские спаны, атрибуты, статус.
4. OpenTelemetry. Semantic conventions for generative AI — стандартные атрибуты спанов вызовов моделей и инструментов.
5. MLflow. Tracing: ручной трейсинг, поиск трейсов, автоматический трейсинг OpenAI-совместимого клиента на примере Ollama.
6. GitHub. Building and testing Python; секреты в GitHub Actions.
7. GitLab. Unit test reports: Python; CI/CD variables — хранение секретов.
8. Beyer B. et al. (eds.). Site Reliability Engineering. O'Reilly, 2016. Гл. 6: Monitoring Distributed Systems — четыре золотых сигнала.

Приложение. Фрагменты для реализации

Трейсинг эпизода:

```python
import mlflow
from mlflow.entities import SpanType

mlflow.openai.autolog()                          # спаны вызовов модели с токенами

def run_episode(agent, suite, task, version, run_idx):
    with mlflow.start_span(name=f"episode {task.ID}", span_type=SpanType.AGENT) as span:
        span.set_inputs({"task": task.ID, "prompt": task.PROMPT})
        utility, _ = suite.run_task_with_pipeline(agent, task, None, {})
        span.set_outputs({"utility": utility})
        mlflow.update_current_trace(tags={"task": task.ID, "version": version,
                                          "run": str(run_idx), "utility": str(utility)})
    return utility

# внутри исполнителя оббязки — спан на каждый вызов инструмента:
#     with mlflow.start_span(name=name, span_type=SpanType.TOOL) as s:
#         s.set_inputs(args); result, error = runtime.run_function(env, name, args)
#         s.set_outputs({"chars": len(output), "error": error})

traces = mlflow.search_traces(filter_string="tag.version = 'base'", flush=True)
```

Гейт — основа решения:

```
def gate(base, cand):   # {task_id: [True, False, True]} — исходы трёх прогонов
    if set(base) != set(cand) or any(len(v) != 3 for v in (*base.values(), *cand.values())):
        return "ERROR", []
    protected = [t for t, runs in base.items() if all(runs)]
    lost = [t for t in protected if sum(cand[t]) < 2]
    if lost:
        return "FAIL", lost
    return ("REVIEW" if len(protected) < 5 else "PASS"), []   # + проверки затрат и остальных задач
```

CI на GitHub Actions — файл `.github/workflows/regression.yml`:

```
name: regression
on: [push, pull_request]
jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v5
        with: {python-version: "3.11", cache: pip}
      - run: pip install -r requirements.txt
      - run: pytest tests/                       # тесты без модели
      - run: python gate.py eval/base/summary.json eval/candidate/summary.json --report
 ↪  gate_report.md
      - uses: actions/upload-artifact@v4
        if: always()
        with: {name: gate-report, path: gate_report.md}
```

В GitLab CI то же самое описывается одной задачей в `.gitlab-ci.yml` с `script:` из тех же команд и `artifacts:` для отчёта [7].